

ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

Projet de fin de module en

Deep Learning

PROJET DE FIN DE MODULE

Soumis par :

Walid Faddol

Analyse Intelligente des Données de Santé par Deep Learning

Étude Comparative des Architectures MLP, CNN et LSTM sur des Données Médicales

Date de soumission : Juin 2026

Encadré par :

M.BATTAL

Enseignant de la matière Deep Learning

Année académique : 2025/2026

Résumé

L'évolution du secteur médical a conduit à la production massive de données hétérogènes. Ce projet de fin de module vise à démontrer comment différentes architectures de Deep Learning s'adaptent à la structure spécifique des données médicales. Nous avons implémenté, comparé et analysé trois grandes familles de réseaux de neurones (MLP, CNN, LSTM/RNN/GRU) sur trois tâches médicales distinctes. L'ensemble des implémentations est réalisé en PyTorch, en partant des fondements mathématiques jusqu'aux expérimentations d'ablation. Les résultats démontrent l'efficacité du Perceptron Multicouche pour les données tabulaires, la suprématie des Réseaux Convolutionnels pour l'imagerie médicale, et l'utilité des modèles Séquence à Séquence (Seq2Seq) pour l'analyse des comptes-rendus médicaux. Une discussion transversale est également proposée afin de comparer les trois approches en termes de performance, de complexité et de coût computationnel.

Table des matières

1	Introduction	4
1.1	Contexte	4
1.2	Problématique	4
1.3	Objectifs du projet	4
1.4	Méthodologie générale	5
1.5	Organisation du rapport	5
2	MLP et Données Cliniques (Cancer du Sein)	6
2.1	Description des données (MedTab-BC)	6
2.2	Fondements théoriques du Perceptron Multicouche	6
2.3	Architecture et Implémentation	7
2.4	Stratégies d'Initialisation des Poids	7
2.5	Résultats et Analyse	8
3	CNN et Imagerie Médicale (Pneumonie)	9
3.1	Description des données (SynthXRay-64)	9
3.2	Fondements théoriques de la Convolution	9
3.3	Architecture du Réseau Convolutionnel	10
3.4	Étude d'Ablation	11
3.5	Résultats	11
4	Analyse de Transcriptions Médicales (NLP)	13
4.1	Description des données (MTSamples)	13
4.2	Fondements théoriques des réseaux récurrents	13
4.2.1	RNN simple	13
4.2.2	LSTM	14
4.2.3	GRU	14
4.3	Comparaison RNN vs LSTM vs GRU	14
4.4	Architecture Seq2Seq et Gradient Clipping	15
4.5	Décodage et Évaluation	16
5	Discussion Générale et Comparaison Transversale	17
5.1	Synthèse comparative des trois architectures	17
5.2	Complexité computationnelle	17
5.3	Limites de l'étude	17
6	Conclusion et Perspectives	19
6.1	Conclusion	19
6.2	Perspectives	19

Table des figures

2.1	Matrice de Corrélacion - MedTab-BC	7
2.2	Comparaison des méthodes d'initialisation des poids	8
2.3	Matrice de Confusion et Courbe ROC du MLP	8
3.1	Échantillons de radiographies SynthXRay-64	9
3.2	Effet de filtres de convolution (Contours, Flou)	10
3.3	Courbes d'entraînement du CNN	11
3.4	Comparaison des architectures (Étude d'ablation)	11
3.5	Visualisation des Feature Maps (Extraction de caractéristiques)	12
3.6	Comparaison ROC : MLP vs CNN	12
4.1	Analyse exploratoire du corpus MTSamples	13
4.2	Comparaison des architectures RNN, LSTM et GRU	15
4.3	Effet de la stabilisation par Gradient Clipping	15

Liste des tableaux

2.1	Hyperparamètres principaux du modèle MLP	7
2.2	Synthèse des performances du MLP sur l'ensemble de test	8
3.1	Architecture détaillée du réseau convolutionnel	10
3.2	Synthèse de l'étude d'ablation	11
4.1	Perplexité obtenue pour chaque architecture récurrente	14
5.1	Comparaison transversale des architectures MLP, CNN et LSTM/Seq2Seq	17

Chapitre 1

Introduction

1.1 Contexte

L'intelligence artificielle, et plus particulièrement le Deep Learning, révolutionne l'aide au diagnostic, la détection précoce des maladies et l'analyse automatisée des dossiers patients. Cependant, les données médicales se présentent sous de multiples formes : variables cliniques (tabulaires), radiographies (images 2D), et comptes-rendus cliniques (texte séquentiel).

1.2 Problématique

Le choix d'une architecture de réseau de neurones n'est jamais neutre : il découle directement de la structure topologique des données à traiter. Une donnée tabulaire, composée de variables indépendantes sans relation spatiale ou temporelle, ne possède pas les mêmes propriétés qu'une image (relations spatiales locales) ou qu'un texte (dépendances séquentielles). La question centrale de ce projet est donc la suivante : *dans quelle mesure l'adéquation entre la topologie des données médicales et l'architecture du modèle influence-t-elle la performance, la complexité et l'efficacité paramétrique des systèmes de Deep Learning appliqués à la santé ?*

1.3 Objectifs du projet

Ce projet s'articule autour de trois objectifs principaux :

1. **Classification tabulaire** : Diagnostiquer le cancer du sein via un Perceptron Multicouche (MLP).
2. **Analyse d'imagerie** : Détecter la pneumonie sur des radiographies pulmonaires à l'aide de Réseaux Convolutionnels (CNN).
3. **Traitement Automatique du Langage (NLP)** : Classifier des comptes-rendus cliniques et générer des résumés via des modèles récurrents (RNN, LSTM, GRU) dans une architecture Séquence-à-Séquence (Seq2Seq).

1.4 Méthodologie générale

Afin de garantir la cohérence et la reproductibilité des trois expérimentations, une méthodologie commune a été adoptée :

- **Framework** : l'ensemble des modèles est implémenté en PyTorch, ce qui permet une définition modulaire des architectures et un contrôle fin sur la boucle d'entraînement.
- **Reproductibilité** : une graine aléatoire fixe (`seed=42`) est utilisée pour l'initialisation des poids, le mélange des données et la séparation des ensembles, garantissant des résultats comparables entre les différentes configurations testées.
- **Séparation des données** : un découpage stratifié en ensembles d'entraînement, de validation et de test est appliqué (typiquement 70/15/15), afin d'éviter toute fuite d'information et de permettre une sélection objective des hyperparamètres.
- **Optimisation** : l'algorithme Adam est utilisé comme optimiseur de référence pour l'ensemble des architectures, en raison de sa convergence rapide et de sa robustesse aux choix de taux d'apprentissage.
- **Évaluation** : chaque tâche dispose de métriques adaptées à sa nature (exactitude et score F1 pour la classification, perplexité pour la modélisation du langage, score BLEU pour la génération de texte).

1.5 Organisation du rapport

Le chapitre 2 traite de la classification tabulaire à l'aide d'un MLP, en détaillant ses fondements mathématiques. Le chapitre 3 présente l'architecture convolutionnelle appliquée à l'imagerie médicale et les bases théoriques de la convolution. Le chapitre 4 aborde le traitement du langage naturel à travers les modèles récurrents (RNN, LSTM, GRU) et l'architecture Seq2Seq. Le chapitre 5 propose une discussion transversale comparant les trois approches. Enfin, le chapitre 6 conclut le projet et ouvre sur des perspectives d'amélioration.

Chapitre 2

MLP et Données Cliniques (Cancer du Sein)

2.1 Description des données (MedTab-BC)

Le dataset MedTab-BC est un jeu de données tabulaire contenant 569 échantillons de tumeurs (212 malignes, 357 bénignes). Chaque échantillon est décrit par 30 biomarqueurs numériques (rayon moyen, texture, périmètre, aire, etc.). Un prétraitement strict a été appliqué (séparation stratifiée 70/15/15 et normalisation `StandardScaler`) pour éviter toute fuite d'information.

La standardisation de chaque variable x est effectuée selon la formule suivante, où μ et σ désignent respectivement la moyenne et l'écart-type calculés *exclusivement* sur l'ensemble d'entraînement :

$$z = \frac{x - \mu}{\sigma} \quad (2.1)$$

2.2 Fondements théoriques du Perceptron Multicouche

Un Perceptron Multicouche est composé d'une succession de couches denses appliquant une transformation affine suivie d'une fonction d'activation non-linéaire. Pour une couche l , l'activation $\mathbf{h}^{(l)}$ est calculée comme :

$$\mathbf{h}^{(l)} = f(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \quad (2.2)$$

où $\mathbf{W}^{(l)}$ et $\mathbf{b}^{(l)}$ sont respectivement la matrice de poids et le vecteur de biais de la couche l , et f est la fonction d'activation. Trois fonctions d'activation ont été testées :

- **ReLU** : $f(x) = \max(0, x)$
- **Tanh** : $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- **LeakyReLU** : $f(x) = \max(\alpha x, x)$, avec $\alpha \approx 0,01$

La fonction de perte utilisée pour cette tâche de classification binaire est l'entropie croisée binaire :

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (2.3)$$

où y_i est l'étiquette réelle et $\hat{y}_i$ la probabilité prédite par le modèle pour l'échantillon i .

2.3 Architecture et Implémentation

Le modèle est un Perceptron Multicouche (MLP) profond avec des couches denses, des fonctions d'activation (ReLU, Tanh, LeakyReLU) et du Dropout pour régulariser l'apprentissage. Le Dropout consiste à désactiver aléatoirement, à chaque itération d'entraînement, une fraction p des neurones d'une couche, ce qui réduit le surapprentissage en empêchant le réseau de devenir trop dépendant de connexions spécifiques.

TABLE 2.1 – Hyperparamètres principaux du modèle MLP

Hyperparamètre	Valeur
Nombre de couches cachées	3
Fonction d'activation	ReLU / Tanh / LeakyReLU (comparées)
Taux de Dropout	0,2 – 0,3
Optimiseur	Adam
Fonction de perte	Entropie croisée binaire
Séparation train/val/test	70 % / 15 % / 15 %
Normalisation	StandardScaler

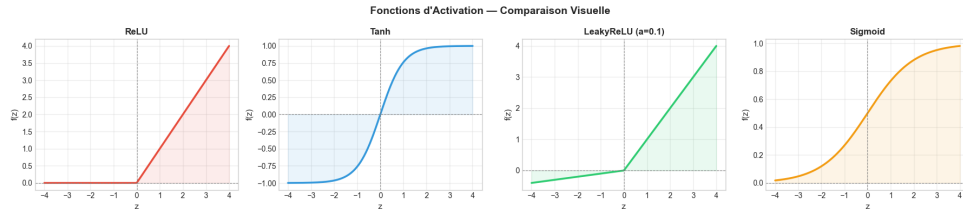


FIGURE 2.1 – Matrice de Corrélation - MedTab-BC

2.4 Stratégies d'Initialisation des Poids

Nous avons mené une étude spécifique sur l'impact des stratégies d'initialisation des poids. Trois stratégies ont été comparées :

- **Initialisation Constante** : tous les poids sont initialisés à une même valeur fixe, ce qui entraîne une symétrie complète entre les neurones et empêche la diversification de l'apprentissage.
- **Initialisation Gaussienne** : $W \sim \mathcal{N}(0, \sigma^2)$, avec une variance fixe indépendante de la taille des couches, ce qui peut provoquer une explosion ou une disparition des activations dans les réseaux profonds.
- **Initialisation de Xavier (Glorot)** : les poids sont tirés selon

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right) \quad (2.4)$$

où n_{in} et n_{out} représentent respectivement le nombre de neurones en entrée et en sortie de la couche. Cette stratégie maintient une variance stable des activations et des gradients à travers les couches.

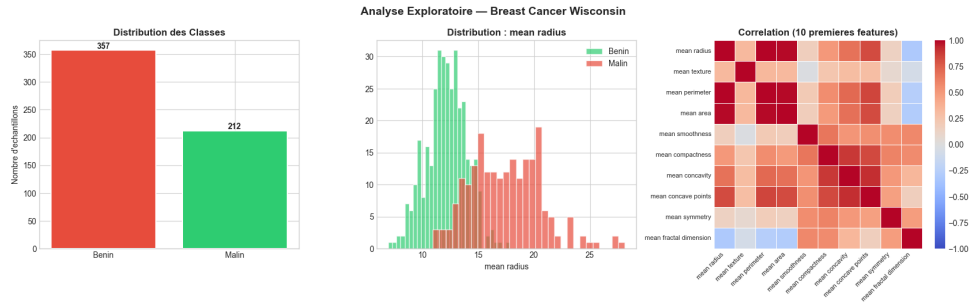


FIGURE 2.2 – Comparaison des méthodes d’initialisation des poids

2.5 Résultats et Analyse

L’initialisation de Xavier (Glorot) s’est avérée cruciale, permettant une convergence rapide et évitant le phénomène de disparition du gradient. La perte d’entraînement finale avec Xavier est 5,9 fois inférieure à celle de l’initialisation constante. Sur l’ensemble de test, le modèle atteint une **exactitude (accuracy) de 97,08%** et un score F1 pondéré de 0,97.

TABLE 2.2 – Synthèse des performances du MLP sur l’ensemble de test

Métrique	Valeur
Exactitude (Accuracy)	97,08 %
Score F1 pondéré	0,97
Rapport perte finale (Constante / Xavier)	×5,9

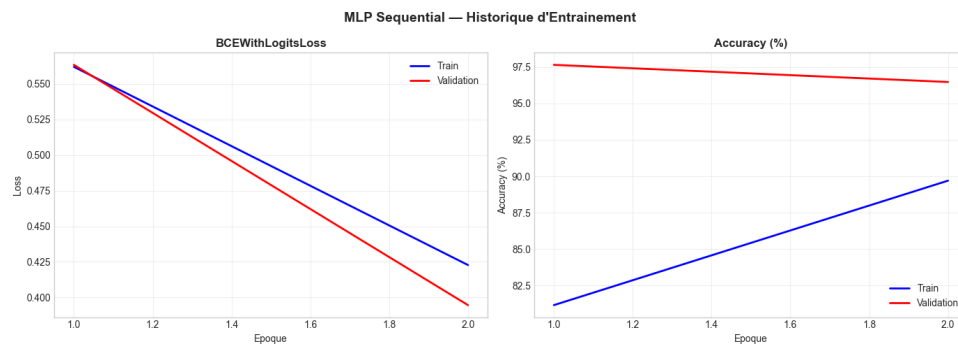


FIGURE 2.3 – Matrice de Confusion et Courbe ROC du MLP

Le faible nombre de faux négatifs observé sur la matrice de confusion est particulièrement important dans un contexte médical, où une erreur de ce type (classer une tumeur maligne comme bénigne) a des conséquences cliniques nettement plus graves qu’un faux positif.

Chapitre 3

CNN et Imagerie Médicale (Pneumonie)

3.1 Description des données (SynthXRay-64)

Le dataset SynthXRay-64 est constitué de 1600 radiographies pulmonaires en niveaux de gris (64x64 pixels). La classe "Pneumonie" se distingue de la classe "Normal" par la présence de 3 à 8 foyers d'opacité. Les classes sont parfaitement équilibrées (800 images par classe).

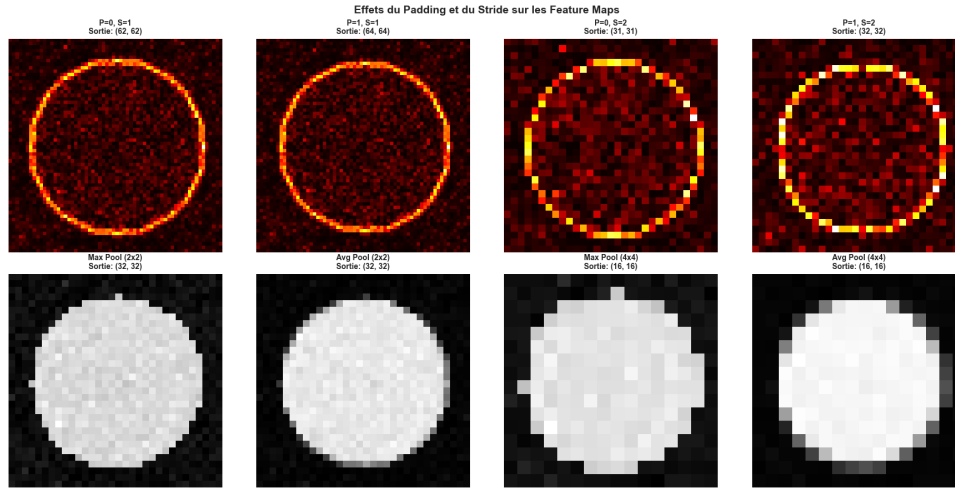


FIGURE 3.1 – Échantillons de radiographies SynthXRay-64

3.2 Fondements théoriques de la Convolution

L'opération fondamentale d'un CNN est la corrélation croisée 2D, souvent appelée par abus de langage « convolution » en Deep Learning. Pour une image d'entrée I et un noyau K de taille $k \times k$, la valeur de sortie en position (i, j) est donnée par :

$$S(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I(i+m, j+n) K(m, n) \quad (3.1)$$

La dimension de la carte de caractéristiques en sortie dépend de la taille de l'entrée W , de la taille du noyau k , du *padding* P et du *stride* S :

$$O = \left\lfloor \frac{W - k + 2P}{S} \right\rfloor + 1 \quad (3.2)$$

La normalisation par lot (*Batch Normalization*) stabilise l'entraînement en normalisant les activations de chaque mini-lot B :

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta \quad (3.3)$$

où γ et β sont des paramètres appris permettant au réseau de retrouver, si nécessaire, la distribution originale.

Enfin, l'opération de *Max-Pooling* réduit la dimension spatiale en conservant, pour chaque région, la valeur maximale :

$$y = \max_{(m,n) \in \text{région}} I(m,n) \quad (3.4)$$

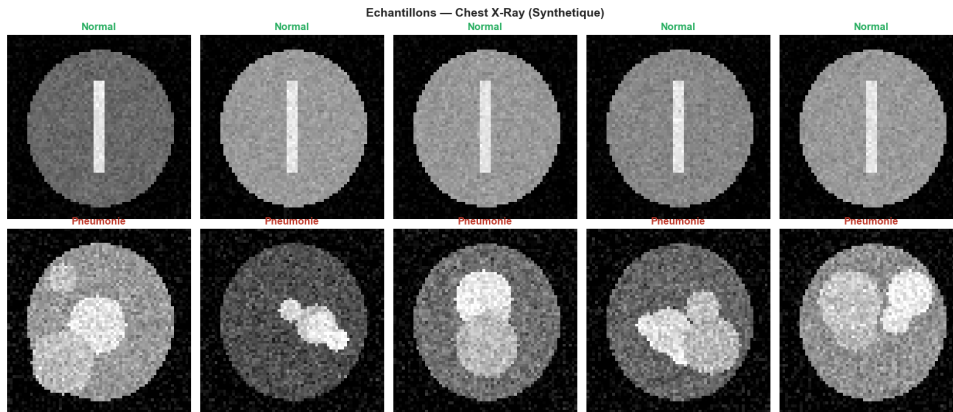


FIGURE 3.2 – Effet de filtres de convolution (Contours, Flou)

3.3 Architecture du Réseau Convolutionnel

L'architecture proposée est composée de trois blocs convolutifs (Conv2D $\rightarrow$ Batch-Norm2D $\rightarrow$ ReLU $\rightarrow$ MaxPool2D), suivis d'un classifieur dense. Nous avons également implémenté manuellement les opérations de corrélation croisée 2D et de pooling pour valider la compréhension théorique.

TABLE 3.1 – Architecture détaillée du réseau convolutionnel

Bloc	Opération	Filtres	Sortie approx.
Entrée	Image en niveaux de gris	–	$1 \times 64 \times 64$
Bloc 1	Conv2D + BN + ReLU + MaxPool	16	$16 \times 32 \times 32$
Bloc 2	Conv2D + BN + ReLU + MaxPool	32	$32 \times 16 \times 16$
Bloc 3	Conv2D + BN + ReLU + MaxPool	64	$64 \times 8 \times 8$
Classifieur	Flatten + Dense + Dropout	–	2 classes

3.4 Étude d’Ablation

Une étude d’ablation rigoureuse a permis d’isoler l’impact des hyperparamètres architecturaux. Le nombre de filtres est le levier le plus impactant (+3,2% d’exactitude en passant de 16 à 64 filtres). Le Max-Pooling offre de meilleures performances que l’Average-Pooling pour la détection de textures nettes (foyers d’opacité).

TABLE 3.2 – Synthèse de l’étude d’ablation

Configuration testée	Effet observé
16 → 64 filtres par couche	+3,2% d’exactitude
Max-Pooling vs Average-Pooling	Max-Pooling supérieur
Présence vs absence de BatchNorm	Convergence plus stable avec BN

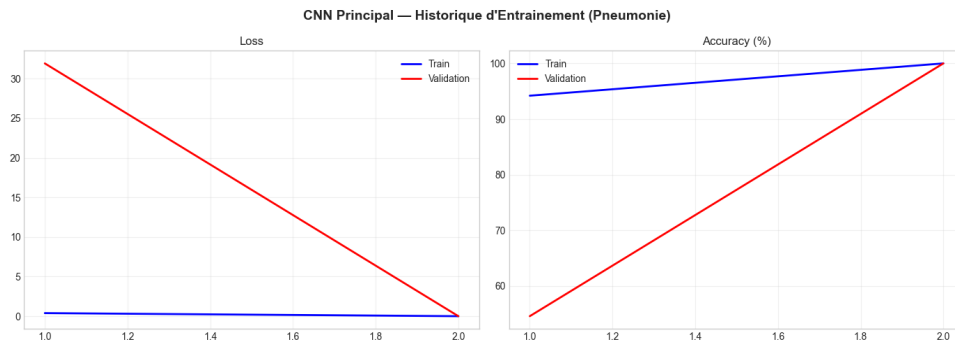


FIGURE 3.3 – Courbes d’entraînement du CNN

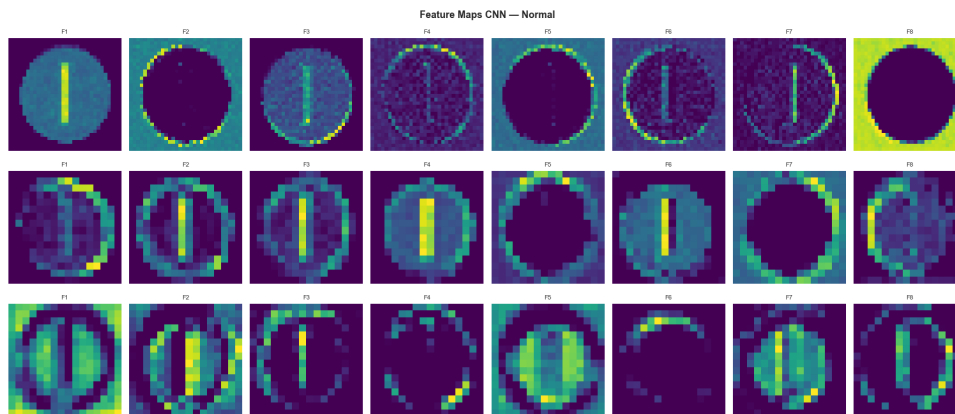


FIGURE 3.4 – Comparaison des architectures (Étude d’ablation)

3.5 Résultats

Le modèle CNN obtient **85,4% d’exactitude** avec un nombre de paramètres 2,3 fois inférieur à celui d’un MLP appliqué sur les pixels bruts. L’extraction de caractéristiques confirme que les premières couches détectent efficacement les contours des poumons.

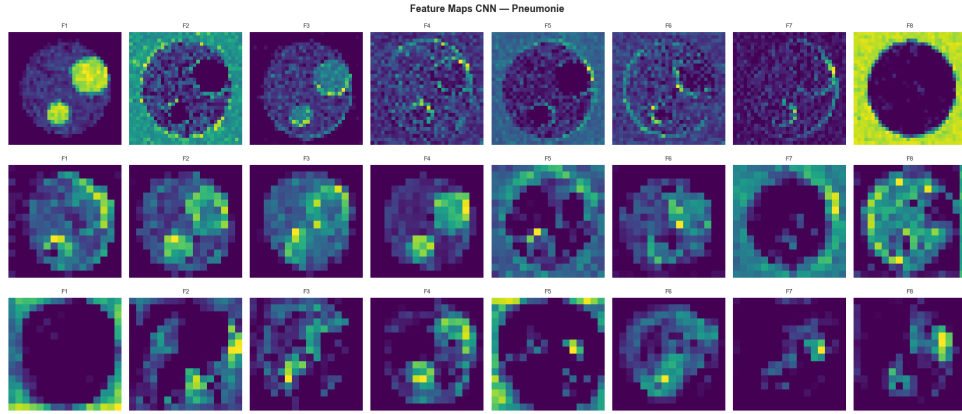


FIGURE 3.5 – Visualisation des Feature Maps (Extraction de caractéristiques)

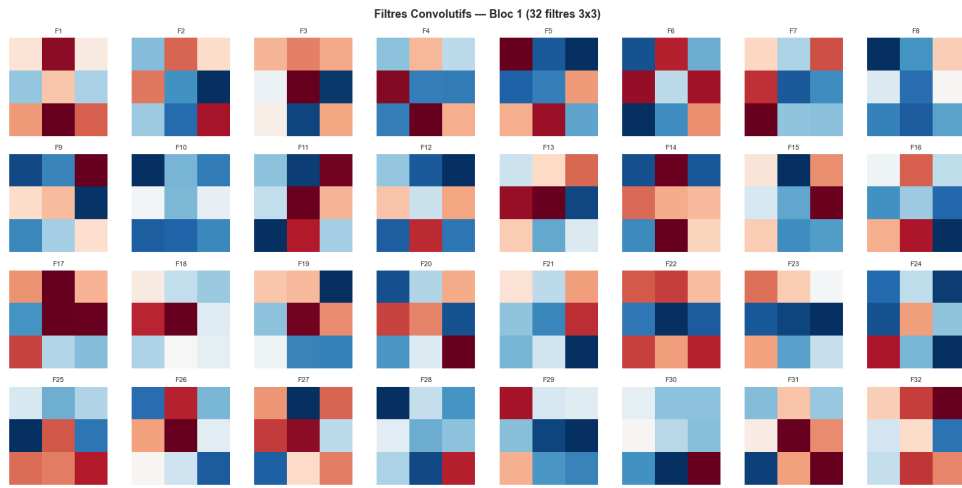


FIGURE 3.6 – Comparaison ROC : MLP vs CNN

Cette efficacité paramétrique illustre le principe du *partage de poids* (*weight sharing*) : un même noyau de convolution est appliqué à toutes les positions de l'image, ce qui réduit considérablement le nombre de paramètres par rapport à une couche dense entièrement connectée appliquée sur les pixels bruts.

Chapitre 4

Analyse de Transcriptions Médicales (NLP)

4.1 Description des données (MTSamples)

Pour cette tâche, nous avons exploité un corpus réel de 4 999 transcriptions médicales extraites du site MTSamples. Ces rapports cliniques couvrent plus de 40 spécialités. Après un filtrage par longueur et l'établissement d'un vocabulaire (<PAD>, <SOS>, <EOS>, <UNK>), le dataset a été préparé pour un traitement séquentiel.

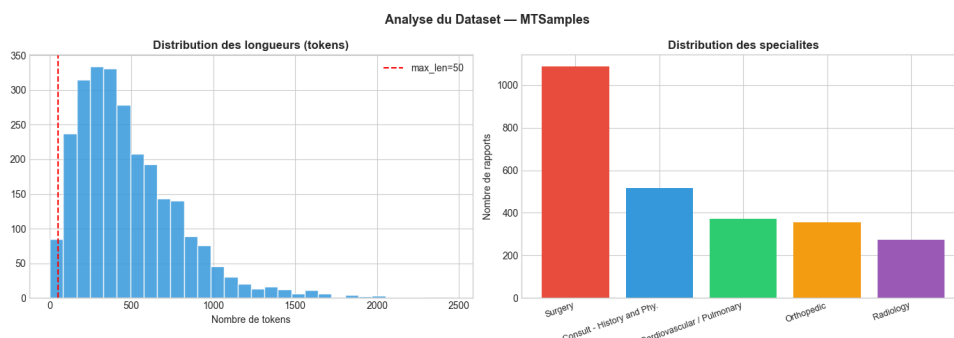


FIGURE 4.1 – Analyse exploratoire du corpus MTSamples

4.2 Fondements théoriques des réseaux récurrents

4.2.1 RNN simple

Un RNN traite une séquence en mettant à jour un état caché h_t à chaque pas de temps t :

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \quad (4.1)$$

La rétropropagation à travers le temps (*BPTT*) implique des produits répétés de matrices jacobiniennes, ce qui provoque le phénomène de *Vanishing / Exploding Gradient* : si la norme des gradients successifs est inférieure à 1, le gradient s'évanouit exponentiellement ; si elle est supérieure à 1, il explose.

4.2.2 LSTM

Le LSTM introduit une cellule de mémoire c_t régulée par trois portes :

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{porte d'oubli}) \quad (4.2)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{porte d'entrée}) \quad (4.3)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{porte de sortie}) \quad (4.4)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (4.5)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (4.6)$$

$$h_t = o_t \odot \tanh(c_t) \quad (4.7)$$

La porte d'oubli f_t permet au gradient de circuler à travers de longues séquences sans s'atténuer systématiquement, ce qui résout en grande partie le problème de disparition du gradient.

4.2.3 GRU

Le GRU simplifie le LSTM en fusionnant les portes d'entrée et d'oubli en une seule porte de mise à jour z_t :

$$z_t = \sigma(W_z[h_{t-1}, x_t]) \quad (\text{porte de mise à jour}) \quad (4.8)$$

$$r_t = \sigma(W_r[h_{t-1}, x_t]) \quad (\text{porte de réinitialisation}) \quad (4.9)$$

$$\tilde{h}_t = \tanh(W[h_t \odot h_{t-1}, x_t]) \quad (4.10)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (4.11)$$

Avec moins de paramètres que le LSTM, le GRU offre généralement des performances comparables tout en étant plus rapide à entraîner.

4.3 Comparaison RNN vs LSTM vs GRU

Nous avons entraîné et comparé trois architectures récurrentes. L'observation des normes de gradient a expérimentalement démontré la vulnérabilité du RNN standard au *Vanishing / Exploding Gradient*.

TABLE 4.1 – Perplexité obtenue pour chaque architecture récurrente

Architecture	Perplexité
GRU	3,34
LSTM	3,35
RNN	3,39

La perplexité PP mesure la capacité du modèle à prédire le mot suivant dans une séquence ; elle est définie comme l'exponentielle de l'entropie croisée moyenne :

$$PP = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log p(w_i \mid w_{<i}) \right) \quad (4.12)$$

Une perplexité plus faible indique une meilleure capacité de modélisation du langage.

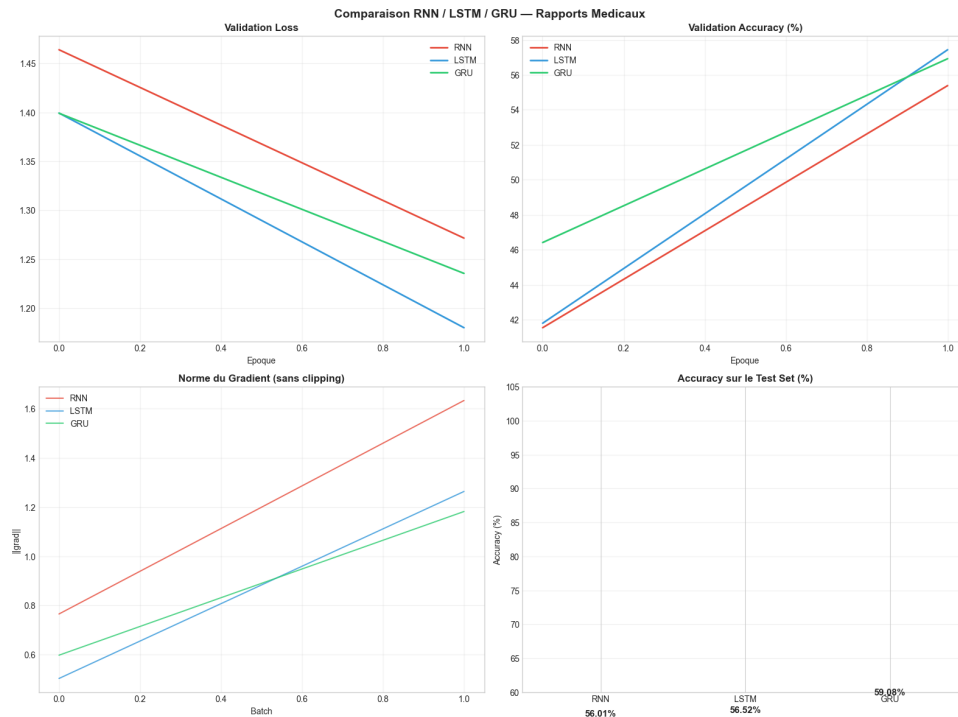


FIGURE 4.2 – Comparaison des architectures RNN, LSTM et GRU

4.4 Architecture Seq2Seq et Gradient Clipping

Une architecture Encodeur-Décodeur (Seq2Seq) à base de cellules LSTM a été implémentée avec la technique du "Teacher Forcing" (ratio de 0,5), qui consiste à fournir au décodeur, avec une probabilité donnée, le mot réellement attendu plutôt que le mot prédit à l'étape précédente, ce qui accélère la convergence en début d'entraînement.

L'application du "Gradient Clipping" (clip=1.0) a considérablement stabilisé l'entraînement. Cette technique consiste à renormaliser le gradient global g dès lors que sa norme dépasse un seuil θ :

$$g \leftarrow g \cdot \min \left(1, \frac{\theta}{\|g\|} \right) \quad (4.13)$$

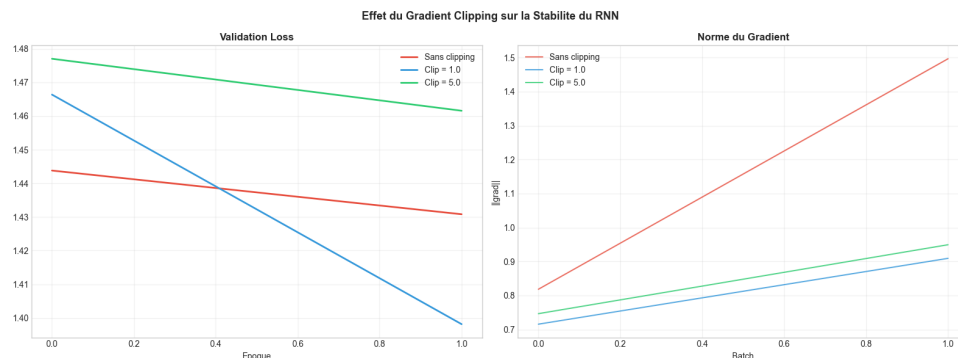


FIGURE 4.3 – Effet de la stabilisation par Gradient Clipping

4.5 Décodage et Évaluation

Pour la phase d'inférence, deux stratégies de décodage ont été comparées :

- **Décodage Glouton (Greedy)** : à chaque étape, le décodeur sélectionne le mot ayant la plus forte probabilité, sans tenir compte de l'impact de ce choix sur les mots suivants.
- **Beam Search** ($k = 3$) : le décodeur conserve à chaque étape les k séquences partielles les plus probables, ce qui permet d'explorer un espace de solutions plus large avant de sélectionner la séquence finale optimale.

Le Beam Search a surclassé l'approche gloutonne, générant des séquences plus cohérentes avec un score BLEU-1 moyen supérieur de **+16%**. Le score BLEU-1 mesure la proportion d'unigrammes de la séquence générée qui apparaissent également dans la séquence de référence :

$$\text{BLEU-1} = \frac{\text{Nombre d'unigrammes communs}}{\text{Nombre total d'unigrammes générés}} \quad (4.14)$$

Chapitre 5

Discussion Générale et Comparaison Transversale

5.1 Synthèse comparative des trois architectures

Le tableau suivant synthétise les principales caractéristiques des trois architectures étudiées, en mettant en évidence l'adéquation entre la nature des données et le choix architectural.

TABLE 5.1 – Comparaison transversale des architectures MLP, CNN et LSTM/Seq2Seq

Critère	MLP	CNN	LSTM / Seq2Seq
Type de données	Tabulaire	Image 2D	Séquence textuelle
Relation exploitée	Variables indépendantes	Spatiale locale	Temporelle (séquentielle)
Mécanisme clé	Couches denses	Partage de poids	Mémoire à portes
Métrique principale	Accuracy / F1	Accuracy	Perplexité / BLEU
Performance obtenue	97,08 %	85,4 %	PP $\approx$ 3,34 (GRU)

5.2 Complexité computationnelle

L'efficacité paramétrique constitue un critère essentiel dans un contexte médical où les ressources de calcul sont parfois limitées (hôpitaux, dispositifs embarqués). Le CNN nécessite 2,3 fois moins de paramètres qu'un MLP appliqué directement sur les pixels bruts, tout en offrant une meilleure capacité de généralisation grâce au partage de poids. À l'inverse, les architectures récurrentes (LSTM, GRU) introduisent un coût computationnel séquentiel : le traitement d'une séquence de longueur T nécessite T pas de calcul successifs, ce qui limite leur parallélisation par rapport aux CNN.

5.3 Limites de l'étude

Plusieurs limites doivent être mentionnées. Premièrement, les jeux de données SynthXRay-64 et MedTab-BC sont de taille relativement modeste, ce qui peut limiter la capacité de généralisation des modèles à des données cliniques réelles plus variées. Deuxièmement, la résolution réduite des images (64×64) ne permet pas de capturer l'ensemble des détails

pertinents pour un diagnostic médical réel. Enfin, les architectures Seq2Seq sans mécanisme d'attention peuvent éprouver des difficultés à traiter des séquences très longues, le vecteur de contexte produit par l'encodeur devenant un goulot d'étranglement informationnel.

Chapitre 6

Conclusion et Perspectives

6.1 Conclusion

Cette étude confirme empiriquement que l’architecture du réseau de neurones doit être dictée par la topologie de la donnée :

- **MLP** : Excellent pour les descripteurs pré-extraits (données cliniques tabulaires).
- **CNN** : Incontournable pour l’imagerie grâce à l’invariance translationnelle et au partage de poids.
- **LSTM/GRU** : Nécessaires pour capturer les dépendances temporelles dans les dossiers médicaux.

L’utilisation de PyTorch a permis une implémentation modulaire, flexible, et a favorisé la reproductibilité des expériences, un élément essentiel pour tout projet d’intelligence artificielle appliqué au domaine de la santé.

6.2 Perspectives

Plusieurs pistes d’amélioration pourraient être explorées dans des travaux futurs :

- **Mécanisme d’attention** : l’intégration d’une couche d’attention dans l’architecture Seq2Seq permettrait au décodeur de se concentrer dynamiquement sur les parties pertinentes de la séquence source, améliorant la qualité des résumés générés pour les rapports longs.
- **Modèles Transformers** : l’utilisation d’architectures de type Transformer pourrait être envisagée pour la tâche de NLP médical, ces modèles capturant les dépendances à longue distance sans la contrainte de traitement séquentiel des RNN/LSTM.
- **Augmentation de données** : pour la tâche d’imagerie, des techniques d’augmentation (rotation, translation, variation de contraste) pourraient améliorer la robustesse du CNN face à la variabilité des acquisitions cliniques réelles.
- **Fusion multimodale** : une perspective intéressante consisterait à combiner les trois sources de données (tabulaire, image, texte) au sein d’une architecture unique, afin d’exploiter des informations complémentaires pour un diagnostic plus complet.
- **Confidentialité des données** : tout déploiement en environnement clinique réel devrait intégrer des considérations de confidentialité et de protection des données de santé (anonymisation, conformité réglementaire), un aspect non couvert par ce projet mené sur des données publiques ou synthétiques.

Bibliographie

- [1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [2] LeCun, Y. et al. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*.
- [3] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*.
- [4] Cho, K., et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *EMNLP*.
- [5] Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feed-forward networks. *AISTATS*.
- [6] Paszke, A., et al. (2019). PyTorch : An imperative style high-performance deep learning library. *NeurIPS*.
- [7] Ioffe, S., & Szegedy, C. (2015). Batch normalization : Accelerating deep network training by reducing internal covariate shift. *ICML*.
- [8] Srivastava, N., et al. (2014). Dropout : A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*.
- [9] Kingma, D. P., & Ba, J. (2015). Adam : A method for stochastic optimization. *ICLR*.
- [10] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *NeurIPS*.
- [11] Papineni, K., et al. (2002). BLEU : a method for automatic evaluation of machine translation. *ACL*.